VoxNote: 로컬 AI 기반 차세대 회의록 서비스

아키텍처 설계 및 구현 방법론

버전: 1.0
작성일: 2026-03-27
설계 철학: Privacy-First, Local-First, Rust-Native

목차

1. 설계 철학 및 티로 대비 차별화 전략
2. 시스템 전체 아키텍처
3. 코어 엔진 레이어 (Rust Native)
4. 크로스플랫폼 배포 전략
5. 서버사이드 아키텍처 (최소 구성)
6. 데이터 흐름 및 파이프라인
7. 모델 관리 및 배포 전략
8. 프로젝트 구조 및 Cargo 워크스페이스
9. 핵심 모듈별 구현 가이드
10. 빌드 및 CI/CD 파이프라인
11. 성능 벤치마크 목표
12. 로드맵

---

1. 설계 철학 및 티로 대비 차별화 전략

1.1 티로(Tiro) 분석 결과 도출된 한계점

티로의 한계	VoxNote 해결 전략
클라우드 의존 (AssemblyAI, OpenAI)	**100% 로컬 추론** — 데이터가 디바이스를 떠나지 않음
오프라인 시 기능 제한	**완전한 오프라인 동작** — 네트워크 없이 STT/요약 가능
월 $7~29 구독 비용	**일회성 구매 or 오픈소스** — 서버 비용 ≈ 0
AI 학습 금지 약속(신뢰 기반)	**기술적으로 불가능한 구조** — 데이터 전송 자체가 없음
웹/데스크탑/모바일 별도 개발	**단일 Rust 코어 + Tauri 2.0** 크로스플랫폼
15개 언어 (클라우드 모델 의존)	**GGUF 양자화 모델 교체**로 언어 무제한 확장

1.2 핵심 설계 원칙

```
┌──────────────────────────────────────────┐
│              DESIGN PRINCIPLES           │
│                                          │
│  1. LOCAL-FIRST   — 모든 AI 추론은 디바이스 내   │
│  2. RUST-NATIVE   — 코어 로직 100% Rust          │
│  3. ZERO-TRUST    — 서버는 사용자 데이터 비접근  │
│  4. PLUGIN-ARCH   — 모델/기능 핫스왑 가능        │
│  5. CROSS-PLAT    — 단일 코드베이스, 6개 타겟    │
│  6. PRIVACY-HARD  — 기술적 보장 (약속 X)         │
└──────────────────────────────────────────┘
```

---

2. 시스템 전체 아키텍처

```
╔═══════════════════════════════════════════════════════════════╗
║                      CLIENT (100% Local)                      ╒
╒   ┌──────────────────────────────────────────────────────┐   ╒
╒   │                Tauri 2.0 Shell                       │   ╒
╒   │   ┌──────────────────────────────────────────────┐   │   ╒
╒   │   │           Frontend (WebView)                 │   │   ╒
╒   │   │     React/Solid + TypeScript + TailwindCSS   │   │   ╒
╒   │   │   ┌─────────┐ ┌─────────┐ ┌──────────────┐   │   │   ╒
╒   │   │   │실시간   │ │노트     │ │ 설정/모델    │   │   │   ╒
╒   │   │   │전사 뷰  │ │에디터   │ │ 관리자       │   │   │   ╒
╒   │   │   └────┬────┘ └────┬────┘ └──────┬───────┘   │   │   ╒
╒   │   └────────┼──────────┼─────────────┼──────────┘   │   ╒
╒   │            │    Tauri IPC (invoke/listen)           │   ╒
╒   │   ┌────────┴──────────┴─────────────┴──────────┐   │   ╒
╒   │   │           Rust Core Engine                  │   │   ╒
╒   │   │  ┌──────────┐ ┌──────────┐ ┌──────────┐    │   │   ╒
╒   │   │  │ STT Engine│ │ LLM Engine│ │ TTS Engine│   │   │   ╒
╒   │   │  │whisper.cpp│ │ llama.cpp │ │ piper/bark│   │   │   ╒
╒   │   │  └────┬─────┘ └────┬─────┘ └────┬─────┘    │   │   ╒
╒   │   │       │            │            │          │   │   ╒
╒   │   │  ┌────┴────────────┴────────────┴────┐     │   │   ╒
╒   │   │  │       Audio Pipeline (cpal + rubato)│   │   │   ╒
╒   │   │  │   Capture → Resample → VAD → Buffer │   │   │   ╒
╒   │   │  └────────────────────────────────────┘     │   │   ╒
╒   │   │                                             │   │   ╒
╒   │   │  ┌────────────────────────────────────┐     │   │   ╒
╒   │   │  │       Storage Layer                 │   │   │   ╒
╒   │   │  │  SQLite(rusqlite) + CRDT Sync + E2EE │   │   │   ╒
╒   │   │  └────────────────────────────────────┘     │   │   ╒
```

```
│   │                                                   │     │
│                                                         │
└─────────────────────────────────────────────────────────┘
                              │ (최소한의 통신)
                              │ • 인증 토큰
                              │ • 푸시 알림
                              │ • 라이선스 검증
                              │ • 모델 다운로드 CDN
                              │ • CRDT 동기화 (선택)
                              ▼
┌─────────────────────────────────────────────────────────────┐
│                    SERVER (최소 구성)                       │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────────┐   │
│  │Auth(JWT) │ │Push/FCM  │ │License   │ │Model CDN     │   │
│  │Axum+OIDC │ │Notify    │ │Verify    │ │R2/S3         │   │
│  └──────────┘ └──────────┘ └──────────┘ └──────────────┘   │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  선택적: CRDT Sync Relay (y-crdt 기반)               │   │
│  │  → 암호화된 CRDT 델타만 중계, 평문 접근 불가         │   │
│  └──────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
```

---

3. 코어 엔진 레이어 (Rust Native)

3.1 Audio Pipeline

오디오 캡처부터 STT 입력까지의 전체 파이프라인을 Rust로 구현한다.

```rust
```

```
// 의사 코드: Audio Pipeline 아키텍처

// 크레이트 의존성:
// cpal       = "0.15"    — 크로스플랫폼 오디오 I/O
// rubato     = "0.15"    — 고품질 리샘플링 (16kHz 변환)
// webrtc-vad = "0.4"     — Voice Activity Detection

pub struct AudioPipeline {
    capture: AudioCapture,      // cpal 기반, 마이크 + 시스템사운드
    resampler: Resampler,       // rubato: 입력 → 16kHz mono f32
    vad: VoiceActivityDetector, // Silero VAD or webrtc-vad
    ring_buffer: RingBuffer,    // lock-free 링버퍼 (STT 엔진으로)
}

// 데이터 흐름:
// Mic/System → cpal callback (실시간)
//   → Ring Buffer (lock-free, crossbeam-channel)
//     → Resample to 16kHz mono f32 (rubato)
//       → VAD gate (음성 구간만 통과)
//         → STT Engine input buffer
```

시스템 사운드 캡처 (플랫폼별 전략):

플랫폼	캡처 방식	Rust 크레이트
macOS	ScreenCaptureKit (macOS 13+) / CoreAudio Aggregate Device	`screencapturekit-rs`, `coreaudio-rs`
Windows	WASAPI Loopback	`cpal` (WASAPI loopback 내장)
Linux	PulseAudio Monitor / PipeWire	`cpal` + `libpulse-binding`
iOS	`AVAudioEngine` via Swift bridge	Tauri mobile plugin (Swift)
Android	`AudioRecord` via Kotlin bridge	Tauri mobile plugin (Kotlin)
Web	`MediaDevices.getUserMedia()` + `AudioWorklet`	JS → Tauri IPC

3.2 STT 엔진 (whisper.cpp 바인딩)

```rust
```

```rust
// 크레이트: whisper-rs = "0.16"  (whisper.cpp Rust bindings)
// 지원: CUDA, Metal, Vulkan, CPU(AVX2/NEON)

pub struct SttEngine {
    ctx: Arc<WhisperContext>,    // 모델 로드 (Send + Sync)
    config: SttConfig,
}

pub struct SttConfig {
    model_path: PathBuf,        // ggml-large-v3-turbo-q5_0.bin
    language: String,            // "auto" | "ko" | "en" | "ja" ...
    translate: bool,             // 실시간 번역 모드
    n_threads: usize,            // CPU 스레드 수
    use_gpu: bool,               // Metal/CUDA 가속
    beam_size: usize,            // 정확도 vs 속도 트레이드오프
    vad_threshold: f32,          // VAD 민감도
    custom_vocabulary: Vec<String>, // 사용자 단어장 (고유명사)
}

// 스트리밍 전사 파이프라인:
// 1. VAD가 음성 감지 → 청크 수집 시작
// 2. 2~3초 단위 슬라이딩 윈도우로 whisper에 전달
// 3. 이전 컨텍스트(prompt)에 직전 전사 결과 포함 → 연속성 확보
// 4. 결과를 프론트엔드로 스트리밍 (Tauri event emit)
```

모델 선택 가이드:

모델	크기 (Q5_0)	VRAM/RAM	실시간 배수 (M4)	정확도	권장 용도
tiny	40MB	250MB	15x	★★☆	모바일/저사양
base	80MB	500MB	10x	★★★	일반 회의
small	250MB	1GB	5x	★★★★	다국어 회의
medium	800MB	2.5GB	2.5x	★★★★☆	고정확도 필요 시
large-v3-turbo	850MB	3GB	3x	★★★★★	전문가용 데스크탑

3.3 LLM 엔진 (llama.cpp 바인딩)

회의록 요약, 문서 생성, Ask 기능 등 모든 텍스트 AI를 로컬 LLM으로 처리한다.

```rust
```

```rust
// 크레이트: llama-cpp-2 (llama.cpp 바인딩)
// 또는 자체 FFI wrapper (더 최신 llama.cpp 추적용)

pub struct LlmEngine {
    model: Arc<LlamaModel>,      // GGUF 모델 (Send + Sync)
    session_pool: SessionPool,   // 세션 재사용 풀
}

pub struct LlmConfig {
    model_path: PathBuf,         // Qwen2.5-7B-Q4_K_M.gguf
    context_length: u32,         // 8192 ~ 32768
    n_gpu_layers: i32,           // GPU 오프로드 레이어 수
    n_threads: usize,
    temperature: f32,
    top_p: f32,
    grammar: Option<String>,     // GBNF 문법 (JSON 출력 강제 등)
}

// 핵심 기능별 프롬프트 전략:
// 1. 실시간 요약: 1~2분 전사 텍스트 → 구조화된 요약
// 2. 템플릿 문서: 전체 전사 + 템플릿 지시어 → 양식 문서
// 3. Ask VoxNote: RAG 방식 — 전사 텍스트 청크 → 임베딩 → 검색 → 답변
// 4. 번역: 전사 텍스트 → 대상 언어 번역
```

로컬 LLM 모델 추천:

모델	크기 (Q4_K_M)	RAM	용도	다국어
Qwen2.5-3B	2GB	4GB	모바일/저사양 요약	★★★★
Qwen2.5-7B	4.5GB	8GB	데스크탑 기본	★★★★★
Llama-3.1-8B	5GB	10GB	영어 최적화	★★★
Gemma-2-9B	5.5GB	10GB	균형형	★★★★
Qwen2.5-14B	8.5GB	16GB	고품질 문서 생성	★★★★★

3.4 TTS 엔진 (선택적)

```rust
// 회의 내용 음성 재생, 접근성 지원용
// piper-rs 또는 자체 FFI

pub struct TtsEngine {
    model: PiperModel,          // ONNX 기반 경량 TTS
    config: TtsConfig,
}

// Piper TTS: 20MB 모델로 자연스러운 음성 합성
// 한국어/영어/일본어 모델 각각 제공
// ONNX Runtime 기반으로 CPU에서도 실시간 합성 가능
```

3.5 로컬 저장소

```rust
```

```
// 크레이트:
// rusqlite = "0.32"     — SQLite (FTS5 전문 검색 포함)
// sled or redb          — 고성능 KV store (임시 데이터)
// age (rage)            — 파일 레벨 암호화

pub struct StorageLayer {
    db: SqlitePool,               // 메타데이터, 노트, 설정
    kv: redb::Database,           // 오디오 청크 캐시, 세션 상태
    encryption: AgeEncryption,    // 사용자 키 기반 E2EE
}

// SQLite 스키마 핵심:
// notes        — id, title, created_at, updated_at, folder_id
// transcripts  — id, note_id, timestamp_ms, text, speaker_id, confidence
// summaries    — id, note_id, template_id, content, model_used
// vocabulary   — id, term, replacement, domain (사용자 단어장)
// embeddings   — id, note_id, chunk_idx, vector BLOB (RAG용)

// 전문검색: SQLite FTS5 사용
// CREATE VIRTUAL TABLE transcript_fts USING fts5(text, content=transcripts);
```

---

4. 크로스플랫폼 배포 전략

4.1 Tauri 2.0 기반 통합 아키텍처

```
┌──────────────────────────┐
│  Shared Rust Core        │
│  (voxnote-core)          │
│                          │
```

```
                    │  • AudioPipeline      │
                    │  • SttEngine          │
                    │  • LlmEngine          │
                    │  • StorageLayer       │
                    │  • NoteManager        │
                    └──────────┬────────────┘
                               │
          ┌────────────────────┼────────────────────┐
          │                    │                    │
   ┌──────┴──────┐      ┌──────┴──────┐      ┌──────┴──────┐
   │ Tauri Desktop │    │ Tauri Mobile│      │  Web (WASM)   │
   │             │      │             │      │               │
   │ • macOS     │      │ • iOS       │      │ • wasm-bindgen │
   │ • Windows   │      │ • Android   │      │ • wasm-pack   │
   │ • Linux     │      │             │      │ • WebGPU STT  │
   │             │      │ Swift/Kotlin│      │               │
   │ 직접 FFI    │      │ bridge 필요 │      │ 제한적 모델만 │
   └─────────────┘      └─────────────┘      └─────────────┘
```

4.2 플랫폼별 빌드 타겟 및 특이사항

플랫폼	타겟 트리플	GPU 가속	오디오 캡처	패키징
macOS (Apple Silicon)	`aarch64-apple-darwin`	Metal	CoreAudio + ScreenCaptureKit	`.dmg`, `.app`
macOS (Intel)	`x86_64-apple-darwin`	— (CPU)	CoreAudio	`.dmg`
Windows	`x86_64-pc-windows-msvc`	CUDA / Vulkan	WASAPI Loopback	`.msi`, `.exe` (NSIS)

플랫폼	타겟 트리플	GPU 가속	오디오 캡처	패키징
Linux	`x86_64-unknown-linux-gnu`	CUDA / Vulkan	PulseAudio / PipeWire	`.AppImage`, `.deb`, `.rpm`
iOS	`aarch64-apple-ios`	CoreML	AVAudioEngine (Swift)	`.ipa`
Android	`aarch64-linux-android`	NNAPI / Vulkan	AudioRecord (Kotlin)	`.apk`, `.aab`
Web	`wasm32-unknown-unknown`	WebGPU	MediaDevices API	정적 호스팅

4.3 모바일 전략 상세

모바일에서는 대형 모델 실행이 제한되므로 계층형 전략을 사용한다.

```
┌──────────── 모바일 모델 전략 ────────────┐
│                                          │
│  Tier 1: 온디바이스 (항상 가능)            │
│  ├─ Whisper tiny/base (40~80MB)           │
│  ├─ Qwen2.5-1.5B Q4 (1GB)                 │
│  └─ 기본 요약/메모 기능                    │
│                                          │
│  Tier 2: Wi-Fi 시 로컬 PC 위임             │
│  ├─ 같은 LAN의 VoxNote 데스크탑에 요청     │
│  ├─ mDNS 자동 디스커버리                   │
│  └─ E2EE 터널 통해 대형 모델 활용          │
│                                          │
│  Tier 3: 사용자 선택적 클라우드 (opt-in)   │
│  └─ 사용자가 직접 설정한 API 엔드포인트    │
│      (예: 자체 서버의 Ollama 인스턴스)     │
└──────────────────────────────────────────┘
```

4.4 웹 (WASM) 전략

```rust
```

```rust
// wasm32 타겟 시 조건부 컴파일

#[cfg(target_arch = "wasm32")]
mod wasm_stt {
    // WebGPU + Transformers.js 기반 whisper
    // 또는 whisper.cpp의 WASM 빌드 활용
    // 모델: tiny/base만 (메모리 제한)
}

#[cfg(not(target_arch = "wasm32"))]
mod native_stt {
    // whisper-rs (네이티브 바인딩)
    // 전체 모델 지원
}
```

---

5. 서버사이드 아키텍처 (최소 구성)

서버는 사용자 데이터를 일절 처리하지 않으며, 인증/알림/라이선스/동기화 중계만 담당한다.

5.1 기술 스택

```
Server Stack (Rust-native):
├── Framework:  axum 0.8 + tower
├── Auth:       JWT (RS256) + OAuth2 (Google/Apple OIDC)
├── DB:         PostgreSQL (사용자 계정, 라이선스만)
├── Cache:      Redis (세션, rate limit)
├── Push:       FCM/APNs (firebase-admin-rs)
├── Sync:       y-crdt relay (암호화된 델타만 중계)
```

```
├── CDN:        Cloudflare R2 (모델 파일 배포)
└── Deploy:     fly.io / Railway (최소 인스턴스)
```

5.2 서버 API 엔드포인트 (전체 목록)

```rust
```

```
// 이것이 서버의 전부다. 심플하게 유지한다.

// === 인증 ===
POST   /api/v1/auth/login          // OAuth2 OIDC 로그인
POST   /api/v1/auth/refresh        // JWT 갱신
DELETE /api/v1/auth/logout         // 세션 무효화

// === 라이선스 ===
GET    /api/v1/license/verify      // 라이선스 키 검증
POST   /api/v1/license/activate    // 디바이스 활성화
DELETE /api/v1/license/deactivate  // 디바이스 비활성화

// === 알림 ===
POST   /api/v1/notifications/register   // 푸시 토큰 등록
PUT    /api/v1/notifications/settings   // 알림 설정

// === 동기화 (E2EE CRDT) ===
WS     /api/v1/sync/connect        // WebSocket — 암호화 델타 중계
GET    /api/v1/sync/status         // 동기화 상태 확인

// === 모델 ===
GET    /api/v1/models/catalog      // 사용 가능한 모델 목록
GET    /api/v1/models/:id/download // 모델 다운로드 (R2 signed URL)

// === 사용자 ===
GET    /api/v1/user/profile        // 프로필 조회
PUT    /api/v1/user/profile        // 프로필 수정
DELETE /api/v1/user/account        // 계정 삭제
```

5.3 CRDT 동기화 상세

디바이스 간 노트 동기화는 CRDT(Conflict-free Replicated Data Type)를 사용한다. 서버는 암호화된 바이너리 델타만 중계하며, 평문 데이터에 접근할 수 없다.

```rust
// 크레이트: y-crdt (Yjs Rust port)

// 클라이언트 사이드:
// 1. 노트 변경 → y-crdt 문서에 반영
// 2. 델타 생성 → age(X25519) 암호화
// 3. 암호화된 델타를 서버 WebSocket으로 전송
// 4. 다른 디바이스가 수신 → 복호화 → y-crdt 머지

// 서버 사이드:
// 1. WebSocket으로 암호화된 바이너리 수신
// 2. 수신자 디바이스 목록 조회 (계정 기반)
// 3. 수신자들에게 그대로 중계 (복호화 불가)
// 4. 오프라인 디바이스용 임시 버퍼링 (TTL 30일)
```

---

6. 데이터 흐름 및 파이프라인

6.1 실시간 전사 플로우

```
[마이크/시스템 사운드]
    | cpal callback (48kHz, f32, 스테레오)
    ▼
[Ring Buffer] ─── lock-free (crossbeam)
    |
```

```
    ▼
[Resampler] ─── rubato (48kHz→16kHz, mono)
    |
    ▼
[VAD Gate] ─── Silero VAD / webrtc-vad
    | 음성 감지 구간만 통과
    ▼
[Audio Accumulator]
    | 2~3초 슬라이딩 윈도우
    | + 0.5초 오버랩 (문맥 연속성)
    ▼
[whisper.cpp] ─── GPU 가속 (Metal/CUDA)
    | initial_prompt = 직전 전사 결과 (연속성)
    | + custom vocabulary 주입
    ▼
[Post-Processor]
    | • 오탈자 교정 (LLM 경량 패스)
    | • 고유명사 매칭 (사용자 단어장 Aho-Corasick)
    | • 화자 분리 태깅 (임베딩 기반 클러스터링)
    | • 타임스탬프 정렬
    ▼
[Tauri Event Emit] → 프론트엔드 UI 업데이트
    |
    ▼
[SQLite 저장] + [요약 큐 적재]
```

6.2 요약/문서 생성 플로우

```
[요약 큐] (1~2분 단위 전사 텍스트 축적)
    |
    ▼
```

```
[Prompt Builder]
    | • 시스템 프롬프트 (역할 + 양식 지시)
    | • 이전 요약 컨텍스트 (연속성)
    | • 현재 전사 텍스트
    | • 사용자 커스텀 템플릿 (있으면)
    ▼
[llama.cpp] ─── GGUF Q4_K_M 모델
    | • Streaming token output
    | • GBNF Grammar (JSON/Markdown 강제)
    ▼
[Summary Post-Processor]
    | • 구조 검증
    | • 마크다운 포매팅
    ▼
[UI 업데이트] + [SQLite 저장]
```

6.3 화자 분리 (Speaker Diarization)

```rust
```

```rust
// 접근 방식: 임베딩 기반 온라인 클러스터링
// whisper.cpp 자체에는 화자 분리가 없으므로 별도 파이프라인 구성

pub struct SpeakerDiarizer {
    // 1. 음성 구간(VAD 결과)마다 화자 임베딩 추출
    //    → wespeaker 또는 ECAPA-TDNN (ONNX Runtime)
    // 2. 온라인 클러스터링 (Agglomerative)
    //    → 새 임베딩 → 기존 클러스터와 코사인 유사도 비교
    //    → 임계값 이상이면 기존 화자, 미만이면 새 화자
    // 3. 결과를 전사 세그먼트에 태깅
    embedding_model: OnnxModel,  // ~20MB
    clusters: Vec<SpeakerCluster>,
    threshold: f32,              // 0.65~0.75
}
```

---

7. 모델 관리 및 배포 전략

7.1 모델 레지스트리

```toml
```

```toml
# models/registry.toml — 앱 내장 모델 카탈로그

[[models]]
id = "whisper-large-v3-turbo-q5"
name = "Whisper Large V3 Turbo"
type = "stt"
size_bytes = 891289600
quantization = "Q5_0"
languages = ["auto", "ko", "en", "ja", "zh", "es", "fr", "de"]
min_ram_mb = 3072
gpu_recommended = true
download_url = "https://cdn.voxnote.app/models/whisper/..."
sha256 = "abc123..."

[[models]]
id = "qwen25-7b-q4km"
name = "Qwen 2.5 7B"
type = "llm"
size_bytes = 4831838208
quantization = "Q4_K_M"
context_length = 32768
languages = ["ko", "en", "ja", "zh", "fr", "de", "es"]
min_ram_mb = 8192
gpu_recommended = true
download_url = "https://cdn.voxnote.app/models/llm/..."

[[models]]
id = "speaker-ecapa-tdnn"
name = "ECAPA-TDNN Speaker Embedding"
type = "diarization"
```

```
size_bytes = 20971520
download_url = "https://cdn.voxnote.app/models/diar/..."
```

7.2 모델 다운로드 및 검증

```rust
pub struct ModelManager {
    registry: ModelRegistry,
    download_dir: PathBuf,       // ~/Library/Application Support/VoxNote/model
}

impl ModelManager {
    // 1. 카탈로그 조회 (로컬 캐시 + CDN 최신 동기화)
    // 2. 다운로드 (청크 단위, 재개 가능, 진행률 UI)
    // 3. SHA-256 무결성 검증
    // 4. 양자화 변환 (선택적: 사용자 VRAM에 맞게 재양자화)
    // 5. 모델 핫스왑 (기존 모델 언로드 → 새 모델 로드)

    pub async fn download_model(&self, id: &str,
        progress: impl Fn(f64)) -> Result<PathBuf>;
    pub fn verify_integrity(&self, path: &Path,
        expected_hash: &str) -> Result<bool>;
    pub fn get_recommended_models(&self,
        available_ram: u64, has_gpu: bool) -> Vec<ModelInfo>;
}
```

8. 프로젝트 구조 및 Cargo 워크스페이스

```
voxnote/
├── Cargo.toml                    # Workspace root
├── Cargo.lock
│
├── crates/
│   ├── voxnote-core/             # 핵심 비즈니스 로직 (순수 Rust)
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│   │       ├── audio/
│   │       │   ├── mod.rs
│   │       │   ├── capture.rs    # cpal 오디오 캡처
│   │       │   ├── resample.rs   # rubato 리샘플링
│   │       │   ├── vad.rs        # Voice Activity Detection
│   │       │   └── pipeline.rs   # 통합 파이프라인
│   │       ├── stt/
│   │       │   ├── mod.rs
│   │       │   ├── engine.rs     # whisper-rs 래퍼
│   │       │   ├── streaming.rs  # 실시간 스트리밍 전사
│   │       │   └── postproc.rs   # 후처리 (오탈자, 고유명사)
│   │       ├── llm/
│   │       │   ├── mod.rs
│   │       │   ├── engine.rs     # llama.cpp 래퍼
│   │       │   ├── summarizer.rs # 요약 생성기
│   │       │   ├── templates.rs  # 템플릿 문서 생성
│   │       │   └── rag.rs        # Ask VoxNote (RAG)
│   │       ├── tts/
│   │       │   ├── mod.rs
│   │       │   └── engine.rs     # Piper TTS 래퍼
```

```
│   │       ├── diarize/
│   │       │   ├── mod.rs
│   │       │   └── speaker.rs    # 화자 분리
│   │       ├── storage/
│   │       │   ├── mod.rs
│   │       │   ├── db.rs         # SQLite + FTS5
│   │       │   ├── crypto.rs     # E2EE (age/rage)
│   │       │   └── sync.rs       # CRDT 동기화 클라이언트
│   │       ├── models/
│   │       │   ├── mod.rs
│   │       │   └── manager.rs    # 모델 다운로드/관리
│   │       └── config.rs         # 설정 관리
│   │
│   ├── voxnote-tauri/            # Tauri 앱 (Desktop + Mobile)
│   │   ├── Cargo.toml
│   │   ├── tauri.conf.json
│   │   ├── capabilities/         # Tauri 권한 설정
│   │   ├── src/
│   │   │   ├── main.rs
│   │   │   ├── commands/         # Tauri IPC 커맨드
│   │   │   │   ├── mod.rs
│   │   │   │   ├── recording.rs
│   │   │   │   ├── notes.rs
│   │   │   │   ├── settings.rs
│   │   │   │   └── models.rs
│   │   │   ├── plugins/          # 커스텀 Tauri 플러그인
│   │   │   │   ├── audio_capture.rs
│   │   │   │   └── system_tray.rs
│   │   │   └── state.rs          # 앱 상태 관리
│   │   ├── gen/
│   │   │   ├── android/          # Android 프로젝트
│   │   │   └── apple/            # iOS/macOS Xcode 프로젝트
```

```
│   │   └── mobile-plugins/
│   │       ├── android/          # Kotlin 네이티브 플러그인
│   │       │   └── AudioCapturePlugin.kt
│   │       └── ios/              # Swift 네이티브 플러그인
│   │           └── AudioCapturePlugin.swift
│   │
│   ├── voxnote-server/           # 서버 (인증/알림만)
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── main.rs
│   │       ├── routes/
│   │       │   ├── auth.rs
│   │       │   ├── license.rs
│   │       │   ├── notification.rs
│   │       │   ├── sync_relay.rs
│   │       │   └── models.rs
│   │       ├── middleware/
│   │       │   ├── auth_guard.rs
│   │       │   └── rate_limit.rs
│   │       └── db.rs
│   │
│   └── voxnote-wasm/             # Web(WASM) 타겟
│       ├── Cargo.toml
│       └── src/
│           ├── lib.rs
│           └── wasm_bridge.rs    # wasm-bindgen 인터페이스
│
├── frontend/                      # 공유 프론트엔드
│   ├── package.json
│   ├── vite.config.ts
│   ├── src/
│   │   ├── App.tsx
```

```
│   │   ├── components/
│   │   │   ├── LiveTranscription.tsx
│   │   │   ├── NoteEditor.tsx
│   │   │   ├── SummaryPanel.tsx
│   │   │   ├── ModelManager.tsx
│   │   │   ├── AskVoxNote.tsx
│   │   │   └── Settings.tsx
│   │   ├── hooks/
│   │   │   ├── useRecording.ts
│   │   │   ├── useStt.ts
│   │   │   └── useLlm.ts
│   │   ├── stores/               # 상태 관리 (zustand)
│   │   └── lib/
│   │       └── tauri-bridge.ts   # Tauri IPC 래퍼
│   └── index.html
│
├── models/                        # 모델 레지스트리 & 스크립트
│   ├── registry.toml
│   └── download.sh
│
└── .github/
    └── workflows/
        ├── ci.yml                 # 테스트 + 린트
        ├── build-desktop.yml      # macOS/Windows/Linux 빌드
        ├── build-mobile.yml       # iOS/Android 빌드
        └── release.yml            # 자동 릴리즈
```

toml

```toml
# Cargo.toml (Workspace Root)
[workspace]
members = [
    "crates/voxnote-core",
    "crates/voxnote-tauri",
    "crates/voxnote-server",
    "crates/voxnote-wasm",
]
resolver = "2"

[workspace.dependencies]
tokio = { version = "1.43", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
anyhow = "1.0"
thiserror = "2.0"
tracing = "0.1"
tracing-subscriber = "0.3"
whisper-rs = { version = "0.16", features = ["metal", "cuda"] }
rusqlite = { version = "0.32", features = ["bundled", "fts5"] }
cpal = "0.15"
rubato = "0.15"
```

---

9. 핵심 모듈별 구현 가이드

9.1 Tauri IPC 커맨드 예시

```rust
```

```rust
// crates/voxnote-tauri/src/commands/recording.rs

use voxnote_core::{AudioPipeline, SttEngine, SttConfig};
use tauri::{command, AppHandle, State, Emitter};

#[command]
pub async fn start_recording(
    app: AppHandle,
    state: State<'_, AppState>,
    config: RecordingConfig,
) -> Result<String, String> {
    let session_id = uuid::Uuid::new_v4().to_string();

    let pipeline = state.audio_pipeline.clone();
    let stt = state.stt_engine.clone();

    // 별도 스레드에서 오디오 캡처 + STT 루프 시작
    tokio::spawn(async move {
        pipeline.start_capture().await;

        loop {
            if let Some(audio_chunk) = pipeline.next_chunk().await {
                match stt.transcribe_streaming(&audio_chunk).await {
                    Ok(segments) => {
                        // 프론트엔드로 실시간 전사 결과 전송
                        let _ = app.emit("stt:segment", &segments);
                    }
                    Err(e) => tracing::error!("STT error: {e}"),
                }
            }
        }
    });
```

```rust
    Ok(session_id)
}

#[command]
pub async fn stop_recording(
    state: State<'_, AppState>,
    session_id: String,
) -> Result<NoteId, String> {
    state.audio_pipeline.stop_capture().await;
    let note = state.note_manager.finalize_note(&session_id).await
        .map_err(|e| e.to_string())?;
    Ok(note.id)
}

#[command]
pub async fn generate_summary(
    state: State<'_, AppState>,
    note_id: String,
    template_id: Option<String>,
) -> Result<String, String> {
    let transcript = state.storage.get_full_transcript(&note_id).await
        .map_err(|e| e.to_string())?;

    let summary = state.llm_engine.summarize(
        &transcript,
        template_id.as_deref(),
    ).await.map_err(|e| e.to_string())?;

    state.storage.save_summary(&note_id, &summary).await
        .map_err(|e| e.to_string())?;
```

```
    Ok(summary)
}
```

9.2 프론트엔드 Tauri 브릿지 예시

```typescript
```

```typescript
// frontend/src/hooks/useRecording.ts

import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';
import { useState, useEffect, useCallback } from 'react';

interface Segment {
  text: string;
  start_ms: number;
  end_ms: number;
  speaker_id?: string;
  confidence: number;
}

export function useRecording() {
  const [isRecording, setIsRecording] = useState(false);
  const [segments, setSegments] = useState<Segment[]>([]);
  const [sessionId, setSessionId] = useState<string | null>(null);

  useEffect(() => {
    const unlisten = listen<Segment[]>('stt:segment', (event) => {
      setSegments(prev => [...prev, ...event.payload]);
    });
    return () => { unlisten.then(fn => fn()); };
  }, []);

  const startRecording = useCallback(async (config: RecordingConfig) => {
    const id = await invoke<string>('start_recording', { config });
    setSessionId(id);
    setIsRecording(true);
    setSegments([]);
  }, []);
```

```
  const stopRecording = useCallback(async () => {
    if (!sessionId) return;
    const noteId = await invoke<string>('stop_recording', { sessionId });
    setIsRecording(false);
    return noteId;
  }, [sessionId]);

  return { isRecording, segments, startRecording, stopRecording };
}
```

---

10. 빌드 및 CI/CD 파이프라인

10.1 GitHub Actions 매트릭스 빌드

```yaml
```

```yaml
# .github/workflows/build-desktop.yml
name: Build Desktop
on:
  push:
    tags: ['v*']

jobs:
  build:
    strategy:
      matrix:
        include:
          - os: macos-latest
            target: aarch64-apple-darwin
            features: "metal"
          - os: macos-13       # Intel Mac
            target: x86_64-apple-darwin
            features: ""
          - os: windows-latest
            target: x86_64-pc-windows-msvc
            features: "cuda"
          - os: ubuntu-latest
            target: x86_64-unknown-linux-gnu
            features: "cuda"

    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
        with:
          targets: ${{ matrix.target }}

      - name: Install dependencies (Linux)
```

```yaml
      if: runner.os == 'Linux'
      run: |
        sudo apt-get update
        sudo apt-get install -y \
          libwebkit2gtk-4.1-dev libappindicator3-dev \
          libasound2-dev libpulse-dev cmake

    - name: Build Tauri App
      run: |
        cd crates/voxnote-tauri
        cargo tauri build --target ${{ matrix.target }} \
          --features ${{ matrix.features }}

    - name: Upload Artifacts
      uses: actions/upload-artifact@v4
      with:
        name: voxnote-${{ matrix.target }}
        path: target/${{ matrix.target }}/release/bundle/
```

10.2 모바일 빌드

```yaml
```

```yaml
# .github/workflows/build-mobile.yml
jobs:
  android:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-java@v4
        with: { java-version: '17' }
      - name: Setup Android SDK/NDK
        uses: android-actions/setup-android@v3
      - run: |
          rustup target add aarch64-linux-android
          cd crates/voxnote-tauri
          cargo tauri android build

  ios:
    runs-on: macos-latest
    steps:
      - run: |
          rustup target add aarch64-apple-ios
          cd crates/voxnote-tauri
          cargo tauri ios build
```

---

11. 성능 벤치마크 목표

11.1 티로 대비 목표 수치

지표	티로 (추정)	VoxNote 목표	비고
전사 지연 (발화→텍스	~500ms	**< 300ms**	로컬 GPU 활용 시

지표	티로 (추정)	VoxNote 목표	비고
트)			
앱 시작 시간	2~3초 (웹 로딩)	**< 1초**	네이티브 Tauri
메모리 사용량	N/A (웹 브라우저)	**< 500MB** (기본)	STT 모델 로드 시 +모델 크기
바이너리 크기	N/A (웹)	**< 30MB** (모델 미포함)	Tauri 경량성
오프라인 가용성	제한적	**100%**	핵심 차별점
데이터 프라이버시	약속 기반	**기술적 보장**	데이터 전송 없음
1시간 회의 STT	서버 의존	**< 8분** (CPU) / **< 2분** (GPU)	파일 업로드 모드
요약 생성 속도	~3초 (서버)	**< 10초** (7B 로컬)	모델 크기 의존

11.2 플랫폼별 최소 요구사양

플랫폼	CPU	RAM	저장공간	GPU (선택)
macOS	Apple M1+	8GB	5GB	Metal (내장)
Windows	4코어 이상	8GB	5GB	CUDA (GTX 1060+)
Linux	4코어 이상	8GB	5GB	CUDA / Vulkan
iOS	A15+ (iPhone 13+)	—	2GB	CoreML
Android	Snapdragon 8 Gen 1+	6GB	2GB	NNAPI

12. 로드맵

Phase 1: Foundation (Month 1~3)

- [ ] Cargo 워크스페이스 구조 셋업
- [ ] Audio Pipeline (cpal + rubato + VAD)
- [ ] whisper-rs 통합 및 스트리밍 전사
- [ ] SQLite 저장소 레이어
- [ ] Tauri 2.0 데스크탑 셸 (macOS/Windows/Linux)
- [ ] 기본 실시간 전사 UI

Phase 2: Intelligence (Month 4~6)

- [ ] llama.cpp 통합 및 요약 엔진
- [ ] 템플릿 기반 문서 생성
- [ ] 화자 분리 (ECAPA-TDNN)
- [ ] 사용자 단어장 및 고유명사 교정
- [ ] 모델 매니저 (다운로드/교체)
- [ ] Ask VoxNote (RAG)

Phase 3: Platform (Month 7~9)

- [ ] iOS / Android Tauri 모바일 빌드
- [ ] 모바일 네이티브 오디오 플러그인 (Swift/Kotlin)
- [ ] WASM 웹 버전 (경량 모드)
- [ ] 서버: 인증(OAuth2) + 라이선스
- [ ] CRDT 동기화 (디바이스 간)

Phase 4: Polish (Month 10~12)

- [ ] 시스템 사운드 캡처 (화상회의 녹음)
- [ ] 팀 기능 (팀 폴더, 공유 템플릿)
- [ ] 푸시 알림 연동
- [ ] 외부 연동 (Notion, Slack, Confluence 내보내기)
- [ ] 성능 최적화 및 벤치마킹
- [ ] 보안 감사 및 퍼징 테스트

부록 A: 핵심 Cargo 의존성 정리

```toml
```

```toml
[dependencies]
# === AI 엔진 ===
whisper-rs = { version = "0.16", features = ["metal", "cuda"] }
# llama.cpp는 build.rs에서 직접 빌드 + bindgen (최신 추적용)
ort = "2.0"                        # ONNX Runtime (화자 분리, TTS)

# === 오디오 ===
cpal = "0.15"                      # 크로스플랫폼 오디오 I/O
rubato = "0.15"                    # 리샘플링
hound = "3.5"                      # WAV 파일 읽기/쓰기
webrtc-vad = "0.4"                 # Voice Activity Detection

# === 저장소 ===
rusqlite = { version = "0.32", features = ["bundled", "fts5"] }
redb = "2.4"                       # 임베디드 KV store

# === 암호화 ===
age = "0.11"                       # 파일 암호화 (X25519)
argon2 = "0.5"                     # 키 파생
chacha20poly1305 = "0.10"          # 스트리밍 암호화

# === 동기화 ===
yrs = "0.21"                       # y-crdt Rust 구현

# === 텍스트 처리 ===
aho-corasick = "1.1"               # 고유명사 매칭
unicode-segmentation = "1.12"      # 유니코드 토큰화

# === 네트워킹 (클라이언트) ===
reqwest = { version = "0.12", features = ["rustls-tls", "stream"] }
tokio-tungstenite = "0.26"         # WebSocket (동기화)
```

```toml
# === 유틸리티 ===
tokio = { version = "1.43", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
uuid = { version = "1.12", features = ["v4"] }
chrono = "0.4"
anyhow = "1.0"
thiserror = "2.0"
tracing = "0.1"
crossbeam-channel = "0.5"         # lock-free 채널
```

부록 B: feature flag 전략

```toml
```

```toml
# crates/voxnote-core/Cargo.toml

[features]
default = ["stt", "llm"]

# AI 엔진
stt = ["whisper-rs"]
llm = []                    # build.rs에서 llama.cpp 직접 빌드
tts = ["ort"]
diarize = ["ort"]

# GPU 가속 (플랫폼별 조건부)
metal = ["whisper-rs/metal"]
cuda = ["whisper-rs/cuda"]
vulkan = ["whisper-rs/vulkan"]

# 플랫폼
desktop = ["cpal"]
mobile = []                 # Tauri mobile plugin으로 오디오 처리
wasm = []                   # WebGPU + 경량 모델만

# 동기화
sync = ["yrs", "tokio-tungstenite"]
```

이 문서는 VoxNote 프로젝트의 초기 아키텍처 설계서이며, 구현 과정에서 기술적 판단에 따라 변경될 수 있다.